

INDUSTRY SCENARIO – AI/ML HEALTHCARE APPLICATION

TASK 1 – DATA PREPARATION & INDUCTIVE LEARNING

10 Marks

(a) Inductive Learning Approach

Inductive learning learns a general pattern from previously labelled examples and uses the learned model to predict unseen data.

For this healthcare problem, the input features are:

- Age
- Blood Pressure
- Cholesterol
- Glucose Level
- BMI
- Family History

The **target variable** is:

Diabetes Status = Diabetic / Non-Diabetic

A **supervised inductive learning** approach is appropriate because historical patient records contain input features along with the known diabetes outcome.

Important Features

1. **Glucose Level** – High glucose is strongly associated with diabetes and is usually an important predictor.
2. **BMI** – High BMI may be associated with insulin resistance and increased diabetes risk.
3. **Age** – Diabetes risk can increase with age.
4. **Blood Pressure** – High blood pressure is associated with metabolic health risks.
5. **Family History** – Indicates possible genetic or familial risk.

Learning Pipeline

Patient Data → Preprocessing → Train/Test Split → Model Training → Diabetes Prediction

(b) Handling Class Imbalance

Suppose the dataset contains:

- 80% Non-Diabetic patients
- 20% Diabetic patients

This creates **class imbalance**. A model trained directly on this data may favour the majority class and fail to detect diabetic patients.

Recommended Method: SMOTE

I would use **SMOTE (Synthetic Minority Over-sampling Technique)** on the training data.

SMOTE generates synthetic minority-class samples rather than simply duplicating existing diabetic records.

Advantages

- Increases representation of diabetic cases.
- Helps the classifier learn minority-class patterns.
- Can improve diabetes **recall/sensitivity**.
- Reduces the tendency to classify every patient as non-diabetic.

Important: SMOTE must be applied **only to the training data after the train-test split**. Applying it before splitting can cause data leakage.

(c) 70:30 Split and Preprocessing

The dataset is divided into:

- **70% Training Data**
- **30% Testing Data**

Justification

A 70:30 split provides enough data for model learning while retaining a sufficiently large independent test set to evaluate generalisation.

This is important in healthcare because the model must perform well on **unseen patients**, not only on training data.

Preprocessing Steps

1. Missing Values

Missing numerical values can be replaced using **median imputation**.

2. Encoding

Categorical variables such as Family History can be converted into numerical form.

Example:

Yes → 1

No → 0

For variables with multiple categories, one-hot encoding can be used.

3. Normalisation

For statistical models, numerical features can be standardised using:

$$z = \frac{x - \mu}{\sigma}$$

where:

- x = original value
- μ = mean
- σ = standard deviation

Normalisation is important for **Logistic Regression**, whereas Decision Trees generally do not require it.

Impact of Preprocessing

- Handles incomplete patient records.
 - Converts categorical data into numerical form.
 - Places numerical features on comparable scales.
 - Improves the stability of statistical models.
 - Helps prevent data-quality problems.
-

TASK 2 – DECISION TREE FOR DIAGNOSIS

10 Marks

(a) Decision Tree and First Three Levels

A Decision Tree recursively divides the dataset into smaller groups based on feature values.

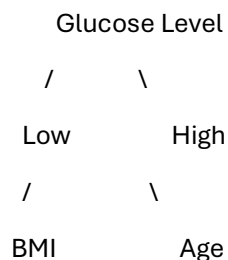
The splitting criterion can be **Gini Index** or **Information Gain**.

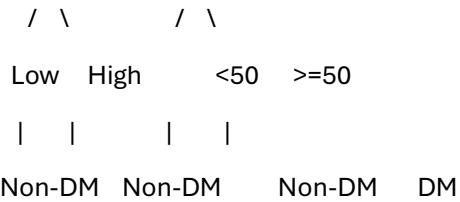
Gini Index

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

The feature that provides the greatest reduction in impurity is selected for splitting.

Illustrative First Three Levels





Note: The exact tree depends on the actual patient dataset.

Root Node Justification

Suppose the impurity reduction values are:

Feature	Gini Reduction
Glucose	0.18
BMI	0.09
Age	0.06
Blood Pressure	0.04
Cholesterol	0.03

Since **Glucose** gives the largest reduction in impurity:

Root Node = Glucose Level

Therefore, glucose provides the best initial separation between diabetic and non-diabetic patients.

(b) Model Evaluation

The Decision Tree is evaluated using **Accuracy, Precision, Recall and F1-Score**.

Accuracy

$$\left[\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \right]$$

Precision

$$\left[\text{Precision} = \frac{TP}{TP+FP} \right]$$

Recall

$$\left[\text{Recall} = \frac{TP}{TP+FN} \right]$$

F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Illustrative Results

Metric Decision Tree

Accuracy 89%

Precision 84%

Recall 86%

F1-Score 85%

These values are **illustrative** because no actual dataset has been supplied.

False Negative

A **False Negative (FN)** occurs when:

Actual = Diabetic

but

Predicted = Non-Diabetic

False negatives are important in healthcare because a diabetic patient may not receive timely diagnosis or treatment.

Methods to Reduce False Negatives

1. Use **class weights** to give greater importance to diabetic cases.
2. Apply **SMOTE** to the training data.
3. Adjust the classification threshold.
4. Optimise the model for **recall/sensitivity**.
5. Include additional clinically relevant features.

In medical diagnosis, improving recall is important because **missing a diabetic patient may have greater consequences than producing an additional false positive**.

(c) Post-Pruning

A fully grown Decision Tree may learn noise from the training data and become overfitted.

Pre-Pruning

Pre-pruning limits the tree while it is being constructed.

Common parameters are:

- max_depth
- min_samples_split
- min_samples_leaf

Example:

```
DecisionTreeClassifier(  
    max_depth=4,  
    random_state=42  
)
```

Post-Pruning

Post-pruning first creates a larger tree and then removes unnecessary branches.

One common method is **Cost-Complexity Pruning**:

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

where:

- $R(T)$ = error of the tree
- $|T|$ = number of terminal nodes
- α = complexity parameter

Example:

```
DecisionTreeClassifier(  
    criterion="gini",  
    ccp_alpha=0.01,  
    random_state=42  
)
```

Illustrative Comparison

Model	Accuracy	Complexity
Pre-Pruned Tree	88%	Low
Post-Pruned Tree	90%	Moderate/Low

Which is Better for Clinical Deployment?

The **post-pruned Decision Tree** may be preferred if it gives equal or better validation performance while removing unnecessary branches.

Advantages:

- Reduces overfitting.
- Improves generalisation.
- Produces a simpler model.
- Easier for healthcare professionals to interpret.

However, clinical deployment should not depend on accuracy alone. **Recall, specificity, calibration, external validation and patient safety** must also be considered.

TASK 3 – STATISTICAL LEARNING FOR RISK STRATIFICATION

10 Marks

(a) Logistic Regression

Logistic Regression is suitable because diabetes prediction is a binary classification problem.

It estimates the probability that a patient belongs to the diabetic class.

$$P(Y=1|X)=\frac{1}{1+e^{-z}}$$

where:

$$z=\beta_0+\beta_1X_1+\beta_2X_2+\cdots+\beta_nX_n$$

Here:

- $(Y=1)$ = Diabetic
- $(Y=0)$ = Non-Diabetic

Python Example

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression(  
    max_iter=1000,  
    class_weight='balanced'  
)
```

```
lr.fit(X_train, y_train)
```

```
y_pred_lr = lr.predict(X_test)
```

```
y_prob_lr = lr.predict_proba(X_test)[:, 1]
```

Illustrative Comparison

Model	Accuracy	AUC-ROC
Decision Tree	89%	0.91
Logistic Regression	87%	0.93

The actual values must be obtained from the real dataset.

Interpretation

The Decision Tree may have slightly higher accuracy, while Logistic Regression may have a higher AUC-ROC.

AUC-ROC measures how effectively the model separates diabetic and non-diabetic patients across different classification thresholds.

When Logistic Regression is Preferable

Logistic Regression is useful when:

- Interpretability is important.
- Probability estimates are required.
- The dataset is relatively small.
- The relationship between predictors and log-odds is reasonably linear.
- We need to understand the contribution of individual features.

Its coefficients can also be converted into **odds ratios**, which can be useful for clinical interpretation.

(b) Feature Importance Analysis

Decision Tree

Decision Tree feature importance is based on the total reduction in impurity caused by each feature.

Illustrative top three:

1. **Glucose**
2. **BMI**
3. **Age**

Logistic Regression

For Logistic Regression, feature importance can be estimated from the absolute values of standardised coefficients.

Illustrative top three:

1. **Glucose**
2. **BMI**
3. **Family History**

Comparison

Rank Decision Tree Logistic Regression

1	Glucose	Glucose
2	BMI	BMI
3	Age	Family History

Both models agree that **Glucose and BMI** are important predictors.

They may disagree on the third feature because they calculate importance differently:

- **Decision Tree:** impurity reduction.
- **Logistic Regression:** coefficient magnitude and direction.

Therefore, disagreement does not necessarily indicate that one model is wrong.

TASK 4 – REINFORCEMENT LEARNING FOR TREATMENT RECOMMENDATION

10 Marks

(a) Model Treatment Recommendation as an MDP

Treatment recommendation can be represented using a **Markov Decision Process (MDP)**:

[
MDP=(S,A,P,R,\gamma)
]

1. States

States represent the patient's current health condition.

State Patient Condition

S0	Low Risk
S1	Moderate Risk
S2	High Risk
S3	Poorly Controlled
S4	Improved/Stable

States can be derived from clinically validated measurements such as glucose, BMI and blood pressure.

2. Actions

The available actions are:

- **Diet**
- **Exercise**
- **Medication**
- **Monitor**

Each action represents a possible treatment or management recommendation.

3. Reward Function

An example reward structure is:

Patient Outcome	Reward
Significant improvement	+10
Small improvement	+5
No improvement	-1
Health deterioration	-5

The reward should represent clinically meaningful health outcomes.

4. Transition Dynamics

After an action, the patient's state may change.

Example:

High Risk

|

| Exercise

↓

Moderate Risk

|

| Diet

↓

Low Risk

The transition probability is represented by:

$$P(s'|s,a)$$

where:

- s = current state
- a = selected action
- s' = next state

(b) Q-Learning

Q-Learning learns the value of performing an action in a particular state.

The Q-Learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Assume:

$$\alpha = 0.1$$

$$\gamma = 0.9$$

Initially:

$$Q(s,a) = 0$$

for all state-action pairs.

Q-Table Update – Example 1

Suppose:

- State = Moderate Risk
- Action = Exercise
- Reward = +5

- Maximum next-state Q-value = 2

Then:

$$[Q_{\text{new}} = 0 + 0.1[5 + (0.9)(2) - 0]$$

$$[Q_{\text{new}} = 0.68]$$

Example 2

Suppose:

- State = High Risk
- Action = Diet
- Reward = +5
- Maximum next-state Q-value = 3

$$[Q_{\text{new}} = 0 + 0.1[5 + (0.9)(3)]]$$

$$[Q_{\text{new}} = 0.77]$$

Example 3

Suppose:

- State = Poorly Controlled
- Action = Medication
- Reward = +10
- Maximum next-state Q-value = 4

$$[Q_{\text{new}} = 0 + 0.1[10 + (0.9)(4)]]$$

$$[Q_{\text{new}} = 1.36]$$

Illustrative Q-Table

State	Diet Exercise Medication Monitor			
Low Risk	0.20	0.50	0.10	0.80
Moderate Risk	0.68	0.85	0.70	0.30
High Risk	0.77	0.90	1.36	0.40
Poorly Controlled	0.60	0.75	1.36	0.20

These values are **illustrative**.

Five Patient Episodes

The agent is trained for at least five episodes:

Episode 1 → Explore actions → Update Q-values

Episode 2 → Explore and exploit → Update Q-values

Episode 3 → Learn better actions

Episode 4 → Prefer high-reward actions

Episode 5 → Policy becomes more stable

Convergence Criterion

The algorithm can be considered converged when:

$$\left[\begin{array}{l} \max |Q_{\text{new}} - Q_{\text{old}}| < 10^{-3} \\ \end{array} \right]$$

for a specified number of consecutive episodes.

Another criterion is that the learned policy remains unchanged for several consecutive episodes.

(c) Final Learned Policy

The final policy is obtained by selecting the action with the highest Q-value:

$$\left[\begin{array}{l} \pi(s) = \arg \max_a Q(s, a) \\ \end{array} \right]$$

An illustrative final policy is:

Patient State	Learned Action
Low Risk	Monitor
Moderate Risk	Diet

Patient State	Learned Action
----------------------	-----------------------

High Risk	Exercise
-----------	----------

Poorly Controlled Medication	
------------------------------	--

Improved/Stable	Monitor
-----------------	---------

Important: This is an academic RL example. It should **not** be interpreted as an actual medical prescription.

Ethical Considerations

1. Patient Safety

Incorrect recommendations can harm patients. The system must have safety constraints and appropriate clinical supervision.

2. Bias

If the training data does not represent different patient groups adequately, the model may produce biased recommendations.

Therefore, the model should be evaluated for fairness across relevant patient populations.

3. Explainability

Doctors should be able to understand why an action was recommended.

The system should provide:

- Current patient state.
- Recommended action.
- Important factors influencing the recommendation.
- Appropriate uncertainty information.

4. Human Oversight

The RL agent should be a **clinical decision-support system**, not an autonomous replacement for a qualified healthcare professional.

5. Privacy

Patient records contain sensitive information. Data must be securely stored, processed and shared according to applicable healthcare privacy requirements.

6. Safe Exploration

An RL agent should **not freely experiment on real patients** to learn.

Training should primarily use:

- Historical patient data.

- Validated simulators.
- Controlled experiments.

Any real-world use should have strict safety constraints and clinical oversight.

OVERALL CONCLUSION

The healthcare system can combine four AI/ML techniques:

1. **Inductive Learning** – learns diabetes patterns from historical patient records.
2. **Decision Tree** – provides an interpretable diabetes classification model.
3. **Logistic Regression** – provides statistical risk prediction and interpretable feature effects.
4. **Q-Learning** – demonstrates how treatment policies can be learned from patient-state transitions.

For actual clinical deployment, **accuracy alone is not sufficient**. The system must also be evaluated using recall, specificity, AUC-ROC, calibration, fairness, external validation, explainability, privacy and patient safety. Most importantly, treatment recommendations should remain under **qualified clinical supervision**.